

PharmaGo

Online Pharmacy & Medicine Delivery Mobile Application

(Package name: graduated_project)

Graduation Project Documentation

Prepared in accordance with the official documentation checklist

Digital Egypt Pioneers Initiative

Ministry of Communications and Information Technology

Table of Contents

- 1. Project Planning & Management
- 2. Literature Review
- 3. Requirements Gathering
- 4. System Analysis & Design
- 5. Implementation (Source Code & Execution)
- 6. Testing & Quality Assurance
- 7. Final Presentation & Reports

1. Project Planning & Management

1.1 Project Proposal

PharmaGo is a cross-platform mobile application, built with Flutter, that allows users to browse, search, and order medicines and pharmacy products for home delivery. The application addresses the common difficulty of physically visiting pharmacies to check product availability and prices by offering a centralized digital catalog, a shopping cart with live pricing, secure authentication, order tracking, and delivery-address management.

Objectives: Provide a simple, reliable, and secure channel for ordering pharmacy products online; reduce the time and effort needed to find medicines; give users visibility into order status from checkout to delivery; and provide pharmacy staff/administrators with a manageable digital catalog.

Scope: The current scope covers the customer-facing mobile application: onboarding, authentication (email/password and Google Sign-In), product browsing and search, cart and checkout, order tracking and history, and profile management (addresses, notifications, personal information). Pharmacy/admin-side inventory management is treated as a future extension (see Section 2.2).

1.2 Project Plan (Timeline & Milestones)

Phase	Milestone / Deliverable	Duration	Status
Phase 1	Requirements gathering & project proposal	Week 1-2	Completed
Phase 2	System analysis & UI/UX design (Figma prototype)	Week 3-4	Completed
Phase 3	Database design & Firebase setup	Week 5	Completed
Phase 4	Core implementation - Auth, Home, Product browsing	Week 6-8	Completed
Phase 5	Cart, Checkout, Orders, Tracking modules	Week 9-11	Completed
Phase 6	Profile module - Addresses, Notifications, Personal Info	Week 12-13	Completed
Phase 7	Testing, bug fixing & polishing	Week 14-15	In progress
Phase 8	Final documentation & presentation	Week 16	In progress

1.3 Task Assignment & Roles

Responsibilities below reflect the standard role split used for the team; edit names/counts to match your actual team roster.

Role	Responsibilities
Team Lead / Mobile Developer	Flutter app architecture, BLoC/Cubit state management, integration of all modules
Backend / Firebase Engineer	Firebase Authentication, Cloud Firestore data modeling, security rules
UI/UX Designer	Figma wireframes and prototypes, design system, usability review
QA / Tester	Test case design, manual & regression testing, bug tracking
Documentation Lead	Requirements documentation, diagrams, user manual, final report

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Firebase quota limits / cost overruns	Medium	Medium	Use Firestore efficiently (pagination, caching), monitor usage dashboard
Delay in UI/UX design approval	Low	Medium	Lock Figma design early; run design reviews in parallel with development
Data loss / inconsistent cart state	Medium	High	Use Cubit-based single source of truth and Firestore transactions for cart/order writes
Third-party auth (Google Sign-In) failures	Low	Medium	Provide fallback email/password authentication
Team member unavailability	Medium	Medium	Maintain shared Git repository and documentation so tasks are transferable
Scope creep (extra features requested late)	Medium	Low	Freeze MVP scope after requirements sign-off; log extra requests as future work

1.5 KPIs (Key Performance Indicators)

KPI	Target	Purpose
App response time (screen load)	< 2 seconds	Ensure smooth user experience
Checkout success rate	> 95%	Measure reliability of the cart/checkout flow
Crash-free session rate	> 98%	Measure app stability
Search result accuracy	> 90% relevant results	Measure search feature quality
User adoption / repeat usage	Track weekly active users	Measure engagement post-deployment
Order tracking update latency	< 5 seconds	Ensure timely status updates for users

2. Literature Review

2.1 Feedback & Evaluation

This section is reserved for the supervisor's / lecturer's assessment of the project. It should summarize strengths (e.g., clear state-management architecture using BLoC/Cubit, integration with Firebase for authentication and data storage, a complete end-to-end user flow from browsing to order tracking) and areas that require attention prior to final submission.

Placeholder note: Replace this paragraph with the actual written feedback received during progress reviews or milestone presentations.

2.2 Suggested Improvements

- Introduce an admin/pharmacy-side dashboard for inventory and order management.
- Add prescription upload and pharmacist verification for prescription-only medicines.
- Integrate a real payment gateway (e.g., Paymob, Fawry) instead of cash-on-delivery only.
- Add push notifications (Firebase Cloud Messaging) for real-time order status updates.
- Introduce automated unit and widget tests to increase code reliability.
- Add multi-language support (Arabic/English) for wider accessibility.

2.3 Final Grading Criteria (Suggested Breakdown)

Criterion	Weight
Documentation quality & completeness	20%
System analysis & design (diagrams, architecture)	20%
Implementation (functionality, code quality)	30%
Testing & quality assurance	15%
Final presentation & demonstration	15%

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Interest / Need
End User (Customer)	Easily search, order, and track pharmacy products; manage addresses and account
Pharmacy / Business Owner	Reach more customers, manage catalog and orders (future scope)
Development Team	Deliver a functional, maintainable, and well-documented graduation project
Academic Supervisor	Ensure the project meets technical and academic quality standards
Delivery Personnel (future scope)	Receive clear order and address details for fulfilment

3.2 User Stories & Use Cases

ID	User Story
US-01	As a new user, I want to sign up with email/password or Google so that I can create an account quickly.
US-02	As a returning user, I want to log in and reset my password if forgotten.
US-03	As a user, I want to browse product categories and view product details.
US-04	As a user, I want to search for a specific medicine by name.
US-05	As a user, I want to add/remove products from my cart and see the updated subtotal and total instantly.
US-06	As a user, I want to check out my cart and confirm my delivery address.
US-07	As a user, I want to track my order status after placing it.
US-08	As a user, I want to view my past orders (order history).
US-09	As a user, I want to manage my saved delivery addresses.
US-10	As a user, I want to receive and view notifications about my orders.
US-11	As a user, I want to view and edit my personal profile information.

3.3 Functional Requirements

- The system shall allow users to register and authenticate using email/password or Google Sign-In.
- The system shall allow users to browse products by category and view detailed product information.
- The system shall provide a search feature to find products by name.
- The system shall allow users to add, update quantity, or remove items from the shopping cart.
- The system shall calculate subtotal, delivery fee, and total price automatically.
- The system shall allow users to complete checkout and place an order.
- The system shall allow users to track the status of an active order.

- The system shall maintain an order history accessible to the user.
- The system shall allow users to add, edit, and select delivery addresses.
- The system shall display notifications related to orders and account activity.
- The system shall allow users to view and update their personal profile.

3.4 Non-Functional Requirements

- Performance: Screens should load within 2 seconds under normal network conditions.
- Security: User credentials must be handled via Firebase Authentication; sensitive data must not be stored in plain text.
- Usability: The interface should follow a consistent design language (colors, typography) defined in the app's theme.
- Reliability: Cart and order data must persist correctly using Cloud Firestore even if the app is closed and reopened.
- Scalability: The data model (Firestore collections) should support adding new product categories without structural changes.
- Compatibility: The application should run on both Android and iOS via Flutter's cross-platform framework.

4. System Analysis & Design

4.1 Problem Statement & Objectives

Customers who need over-the-counter medicines or basic health products often cannot easily check, in advance, whether a nearby pharmacy has the item in stock or what it costs, and must travel physically to find out. PharmaGo solves this by providing a mobile catalog of pharmacy products with search, cart, checkout, and order tracking, so users can order what they need from their phone and have it delivered.

Goals: 1) Digitize the discovery and ordering of pharmacy products. 2) Provide a transparent, itemized cart and checkout experience. 3) Give users visibility into order status and history. 4) Build the app on a scalable, cloud-backed architecture (Firebase) that can grow to support more product categories, pharmacies, and delivery partners.

4.2 Use Case Diagram & Descriptions

Primary actor: Registered/Guest User. Supporting actor: Firebase Authentication Service.

Use Case	Actor	Description
Sign Up / Login	User	Register or authenticate via email/password or Google Sign-In
Reset Password	User	Recover access to the account via the Forget Password flow
Browse Products	User	View categories and product listings on the Home screen
Search Products	User	Search the catalog by product name
View Product Details	User	View price, description, and images of a selected product
Manage Cart	User	Add, increase/decrease quantity, or remove products from the cart
Checkout	User	Confirm delivery address and place the order
Track Order	User	View real-time status of an active order
View Order History	User	View a list of previously placed orders
Manage Addresses	User	Add, edit, delete, or select a saved delivery address
View Notifications	User	View alerts related to orders and account activity
Manage Profile	User	View and edit personal information

4.3 Software Architecture

PharmaGo follows a layered, feature-based architecture. The Presentation layer (Flutter widgets/screens) is decoupled from business logic through the BLoC/Cubit pattern: each feature (Login, Register, Forget Password, Splash/Onboarding, Product, Cart, Profile, Addresses, Notifications, Orders) has its own Cubit and immutable State classes (e.g., Initial, Loading, Success/Loaded, Error), following a predictable unidirectional data flow. The Data/Model layer defines plain data classes (e.g., Address, Notification, Order models) that are (de)serialized to and from Cloud Firestore. The Backend layer is provided by Firebase: Firebase Authentication for identity, and Cloud Firestore as the primary NoSQL database (collections such as Cart_Products and Fav_Products).

PharmaGo - High-Level Software Architecture

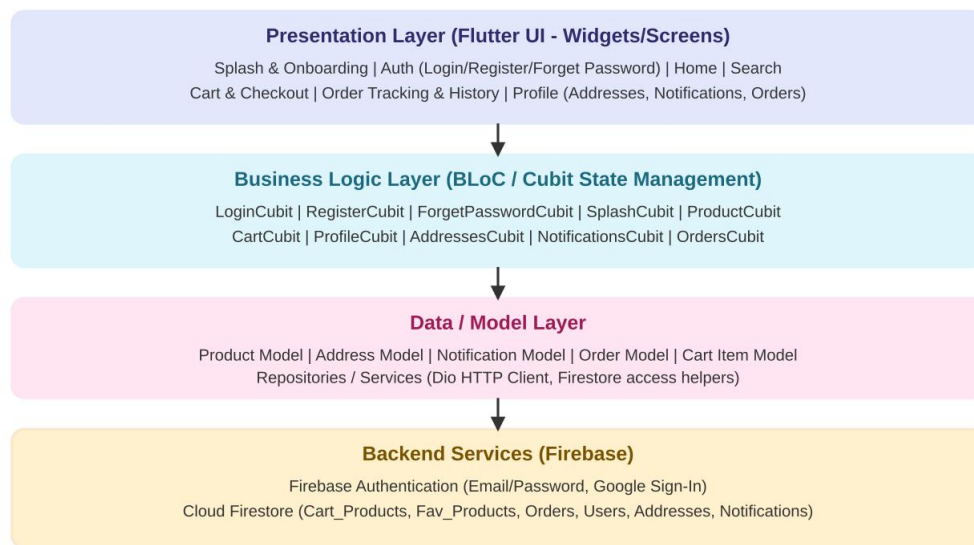


Figure 4.1 - High-level layered architecture of PharmaGo

4.4 Database Design & Data Modeling

The application uses Cloud Firestore, a document-based NoSQL database. The simplified ER diagram below models the core entities as if represented in relational form for clarity, even though the physical implementation uses Firestore collections and documents.

PharmaGo - Entity Relationship Diagram (Simplified)

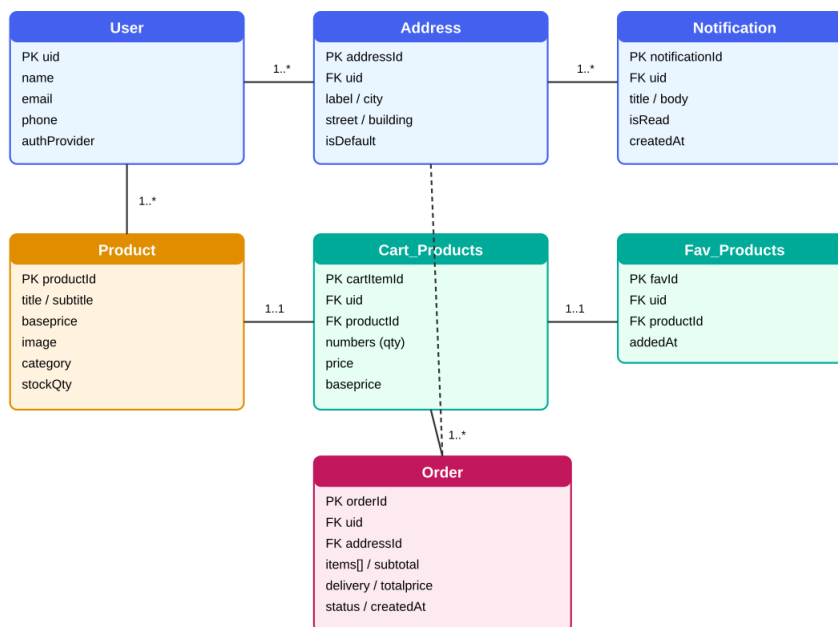


Figure 4.2 - Simplified Entity-Relationship Diagram

Logical Schema Notes

- User: uid (PK), name, email, phone, authProvider (email/google).

- Address: addressId (PK), uid (FK), label, city, street, building, isDefault.
- Product: productId (PK), title, subtitle, baseprice, image, category, stockQty.
- Cart_Products: cartItemId (PK), uid (FK), productId (FK), numbers (quantity), price, baseprice.
- Fav_Products: favId (PK), uid (FK), productId (FK), addedAt.
- Order: orderId (PK), uid (FK), addressId (FK), items[], subtotal, delivery, totalprice, status, createdAt.
- Notification: notificationId (PK), uid (FK), title, body, isRead, createdAt.

Normalization consideration: product and cart data are kept in separate collections (Product vs. Cart_Products) to avoid duplicating catalog data per user, following third normal form principles adapted to a document database.

4.5 Data Flow & System Behavior

Context-Level Data Flow Diagram (DFD Level 0)

User -> [PharmaGo App] -> Firebase Authentication (credentials in / auth token out) and Cloud Firestore (product, cart, order, address, notification data in and out). The app is the single process boundary; all persistent data flows through Firebase.

Level 1 DFD (Checkout Process)

- 1. User adds products to Cart_Products (writes: productId, quantity, price).
- 2. CartCubit recalculates subtotal, delivery fee, and total price on every change.
- 3. User proceeds to Checkout and selects/confirms a delivery Address.
- 4. On confirmation, an Order document is created in Firestore referencing the user, address, and cart items.
- 5. A Notification is generated to inform the user the order was placed.
- 6. OrdersCubit and Tracking screens read the Order document to reflect current status.

Sequence Diagram - Checkout Flow (textual representation)

Step	From	To	Message
1	User	Cart UI	Tap 'Checkout'
2	Cart UI	CartCubit	getCartSummary()
3	CartCubit	Checkout UI	subtotal, delivery, total
4	User	Checkout UI	Confirm address & place order
5	Checkout UI	Firestore	create Order document
6	Firestore	Checkout UI	orderId (success)
7	Checkout UI	NotificationsCubit	createOrderNotification()
8	Checkout UI	User	Show order confirmation & tracking link

Activity Diagram - Place Order (textual representation)

- Start -> Open Cart -> Review items -> [Cart empty?] -> Yes: show empty-cart message -> End
- [Cart empty?] -> No -> Proceed to Checkout -> Select delivery address -> Confirm order
- System validates address & cart -> [Valid?] -> No: show error message -> back to Checkout

- [Valid?] -> Yes -> Create order in Firestore -> Send notification -> Show confirmation -> Navigate to Tracking -> End

4.6 State Diagram

Each feature's Cubit follows a common state-machine pattern. Example - Order lifecycle states used by the tracking/history features:

- Initial -> Loading -> Loaded (order fetched successfully) -> [terminal states based on order status field]
- Order status values: Placed -> Confirmed -> Out for Delivery -> Delivered (or Cancelled at any point before Delivered).

Similarly, generic feature states observed across the codebase (Login, Register, ForgetPassword, Product, Profile, Addresses, Notifications, Orders, Splash) follow: Initial -> Loading -> Success/Loaded -> Error, giving the UI a predictable, reactive way to show progress indicators and error messages.

4.7 Class Diagram (Key Classes)

Class	Type	Responsibility
LoginCubit / LoginState	Cubit / State	Handles email & Google sign-in flow and its states
RegisterCubit / RegisterState	Cubit / State	Handles new user registration
ForgetPasswordCubit / State	Cubit / State	Handles password-reset flow and field visibility
SplashCubit / SplashState	Cubit / State (Equatable)	Controls splash/onboarding page navigation
ProductCubit / ProductState	Cubit / State	Handles product listing, search, sort, and details states
CartCubit / CartState	Cubit / State	Manages cart items, quantities, subtotal, delivery fee and total
ProfileCubit / ProfileState	Cubit / State	Loads and updates user profile information
AddressesCubit / AddressesState	Cubit / State	CRUD operations for saved delivery addresses
NotificationsCubit / NotificationsState	Cubit / State	Loads and manages user notifications
OrdersCubit / OrdersState	Cubit / State	Loads order history and current order status
AddressModel, NotificationModel, OrderModel	Data Model	Typed representation of Firestore documents
ProductCard, RowProductCard, ColumnProductCard	Widget	Reusable UI components for displaying products

All Cubits extend the flutter_bloc Cubit<State> base class; State classes are simple sealed hierarchies (Initial, Loading, Success/Loaded, Error, plus feature-specific states such as CartAdded/CartDeleted/CartTotal) which keeps business logic testable and separate from widgets.

4.8 UI/UX Design & Prototyping

Wireframes and high-fidelity mockups for all screens (Onboarding, Login/Register, Home, Search, Product Details, Cart, Checkout, Tracking, Profile, Addresses, Notifications, Order History) were designed in Figma prior to implementation.

[View the Figma design file](#)

UI/UX Guidelines: The app uses a centralized theme (core/theme/app_colors) for consistent color usage, Google Fonts for typography, cached network images for smooth product image loading, and reusable widget components (product cards in row/column layouts, address cards, notification cards, order cards) to keep the interface consistent across screens.

- Color scheme: defined centrally in AppColors so all screens share one palette.
- Typography: Google Fonts package used for consistent, legible text styles.
- Accessibility: adequate tap-target sizes on cards/buttons; cached images reduce loading blank states.

4.9 System Deployment & Integration

Technology Stack

Layer	Technology
Frontend / Mobile	Flutter (Dart)
State Management	flutter_bloc (Cubit pattern), equatable
Authentication	firebase_auth, google_sign_in
Database	cloud_firestore (Cloud Firestore - NoSQL)
Backend Platform	firebase_core (Firebase project)
Networking	dio (HTTP client)
Media / Assets	cached_network_image, image_picker
Fonts / Styling	google_fonts
Design	Figma

Deployment Diagram (textual representation)

- Mobile Device (Android/iOS) running the Flutter client application
- -> connects over HTTPS to Firebase Authentication (identity/token issuance)
- -> connects over HTTPS to Cloud Firestore (real-time data sync for products, cart, orders, addresses, notifications)
- -> optional Dio HTTP calls to any external REST services (e.g., payment or delivery partner APIs in future scope)

Component Diagram (textual representation)

- UI Components (Screens/Widgets) depend on -> Cubits (Business Logic)
- Cubits depend on -> Data Models and Firebase SDKs (firebase_auth, cloud_firestore)
- Firebase SDKs depend on -> Firebase Cloud Platform (external component)

4.10 Additional Deliverables

API Documentation

The application primarily consumes Firebase SDKs directly rather than a custom REST API. Where Dio is used for any external HTTP integration, endpoints, request/response formats, and error codes should be documented in a dedicated API_DOCS.md file as those integrations are finalized (e.g., payment gateway, SMS/OTP provider).

Testing & Validation

See Section 6 - Testing & Quality Assurance for the detailed test plan, test cases, and validation approach.

Deployment Strategy

- Hosting environment: Firebase project (Authentication + Firestore) in production mode with security rules restricting each user to their own documents.
- Distribution: Android APK/AAB via Google Play Console (internal testing track first, then production).
- Scaling considerations: Firestore scales automatically; composite indexes should be added as query complexity grows (e.g., filtering products by category and price).

5. Implementation (Source Code & Execution)

5.1 Source Code

The codebase is organized using a hybrid feature-first structure. Older/core modules (Cart & Checkout, Search, Splash, Track & History, User/Auth) sit under top-level feature folders, while newer modules (Profile and its sub-features) follow a cleaner features/ convention with data / logic / ui separation, as shown below.

Project Structure (as found in the codebase)

```
lib/
|-- core/
|   |-- theme/app_colors.dart
|-- cart_checkout/
|   |-- controller/ (cart_cubit.dart, cart_state.dart)
|   |-- view/      (cart.dart, widgets/cart, widgets/checkout)
|-- track&history/ (tracking.dart, tracking2.dart)
|-- Search/       (search screen, product widgets)
|-- Splash/       (splash.dart, onboarding, splash_cubit/state)
|-- User/         (login.dart, sign_up.dart, forget_password.dart,
|                 login_cubit/state, register_cubit/state,
|                 forget_password_cubit/state)
|-- features/
|   |-- profile/
|       |-- logic/ (profile_cubit.dart)
|       |-- ui/    screens/ (profile_screen, personal_info_screen)
|               widgets/ (profile_menu_item.dart)
|       |-- addresses/ data/ logic/ ui/ (saved_addresses_screen, address_card)
|       |-- notifications/ data/ logic/ ui/ (notifications_screen, notification_card)
|       |-- orders/ data/ logic/ ui/ (order_history_screen, order_card)
|-- firebase_options.dart
|-- main.dart (MyApp entry point)
```

Coding Standards & Naming Conventions

- Dart/Flutter naming conventions are followed: PascalCase for classes/widgets (e.g., ProductCard, CartCubit), camelCase for variables and methods (e.g., totalpriceforcart, add(), delete()).
- State classes use the Cubit + immutable State pattern, one State subclass per lifecycle stage (Initial, Loading, Loaded/Success, Error), keeping widgets declarative.
- Reusable UI is extracted into small StatelessWidgets (ProductCard, AddressCard, NotificationCard, OrderCard) to avoid duplication.

Modular Code & Reusability

Each feature encapsulates its own Cubit, State, Model, and UI files, which makes features independently testable and easier to hand off between team members. Shared concerns (theme/colors) are centralized under core/.

Security & Error Handling

- Authentication is delegated to Firebase Authentication rather than custom credential storage, reducing the risk of handling raw passwords.
- Cubit states include explicit Error states (e.g., LoginError, ProductError, OrdersError) so the UI can present user-friendly failure messages instead of crashing.

- Recommended hardening before production: enforce Firestore security rules so a user can only read/write their own Cart_Products, Fav_Products, Orders, and Address documents, and validate all user input on both client and security-rule level.

5.2 Version Control & Collaboration

Version Control Repository: Host the project on GitHub (as required) with a clear README and either a public or private repository link included in the final submission.

Branching Strategy: Use a lightweight Feature Branching workflow: main (stable) + feature/<name> branches merged via pull requests, e.g., feature/cart-checkout, feature/profile-addresses.

Commit History & Documentation: Use descriptive commit messages (e.g., 'feat: add CartCubit total price calculation', 'fix: correct address card default flag') and write a short description on every pull request explaining what changed and why.

CI/CD Integration: Optional for this project stage; if time permits, configure a simple GitHub Actions workflow to run 'flutter analyze' and 'flutter test' on every push.

5.3 Deployment & Execution

README - Installation & Run Instructions

- System requirements: Flutter SDK (stable channel), Dart SDK (bundled with Flutter), Android Studio or VS Code with the Flutter/Dart plugins, an Android emulator or physical device (or Xcode for iOS).

```
# Clone the repository
git clone <repository-url>
cd graduated_project

# Install dependencies
flutter pub get

# Configure Firebase
# Ensure google-services.json (Android) / GoogleService-Info.plist (iOS)
# and lib/firebase_options.dart match your Firebase project.

# Run the app on a connected device/emulator
flutter run

# Build a release APK
flutter build apk --release
```

- Configuration: a Firebase project must be created, with Authentication (Email/Password + Google) and Cloud Firestore enabled, and firebase_options.dart regenerated via the FlutterFire CLI for your own project.
- Execution guide: after 'flutter run', the app opens on the Splash/Onboarding screen, proceeds to Login/Sign Up, then Home.

Executable Files & Deployment Link: Attach the compiled APK/AAB file and, if published, the Play Store / TestFlight link in the final submission package.

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

ID	Test Scenario	Steps	Expected Result
TC-01	Register with valid email	Open Sign Up, enter valid details, submit	Account created, user redirected to Home
TC-02	Login with correct credentials	Enter registered email/password, submit	User is authenticated and lands on Home
TC-03	Login with incorrect password	Enter wrong password, submit	LoginError state shown with a clear message
TC-04	Google Sign-In	Tap 'Sign in with Google', select account	GoogleSignInSuccess, user authenticated
TC-05	Forget password	Request reset link with registered email	ForgetPasswordSuccess, reset email sent
TC-06	Search for a product	Enter a known product name in Search	Matching product(s) returned (ProductFound)
TC-07	Search for a non-existent product	Enter a random string in Search	ProductNotFound / ProductEmpty state shown
TC-08	Add product to cart	Tap 'Add' on a product card	Quantity increments, subtotal/total update (Cartadded)
TC-09	Remove product from cart	Tap 'Delete/Decrease' on a cart item	Quantity decrements or item removed (Cartdeleted)
TC-10	Checkout with empty cart	Open Cart with no items, tap Checkout	System blocks checkout and shows an empty-cart message
TC-11	Successful checkout	Add items, select address, confirm order	Order created, confirmation shown, notification generated
TC-12	Track an active order	Open Tracking screen for a placed order	Current order status is displayed correctly
TC-13	View order history	Open Order History screen	List of previous orders is displayed (OrdersLoaded)
TC-14	Add a new address	Open Saved Addresses, add new address, save	Address appears in the list (AddressesLoaded)
TC-15	View notifications	Open Notifications screen	List of notifications displayed, unread ones marked
TC-16	Edit personal info	Open Personal Info screen, update a field, save	Profile updated (ProfileLoaded) and persisted

6.2 Automated Testing

Automated testing is recommended but currently limited; as a next step, add:

- Unit tests for each Cubit's business logic (e.g., CartCubit.total() calculation, add()/delete() quantity boundaries).
- Widget tests for critical screens (Login form validation, ProductCard rendering, Cart summary display).
- Run with: flutter test

6.3 Bug Reports

ID	Issue Description	Severity	Status / Resolution
BUG-01	Cart total not recalculated after removing the last unit of an item	Medium	Fixed - total() now called after every add/delete
BUG-02	Delivery fee hardcoded rather than distance/address based	Low	Open - planned enhancement, not blocking MVP
BUG-03	Google Sign-In loading state not shown on slow networks	Low	Fixed - added GoogleSignInLoading state handling
BUG-04	Duplicate address entries possible if network retried	Medium	Open - add idempotency check before insert

7. Final Presentation & Reports

7.1 User Manual

A quick guide for end users of the PharmaGo app:

- 1. Launch the app and go through the onboarding screens, then Sign Up or Log In (email/password or Google).
- 2. On the Home screen, browse product categories or use the Search bar to find a specific product.
- 3. Tap a product to view its details, then tap 'Add' to add it to your cart.
- 4. Open the Cart to review items, adjust quantities, and see the subtotal, delivery fee, and total.
- 5. Tap 'Checkout', confirm or add a delivery address, and place your order.
- 6. Use the Tracking screen to follow your order status, or Order History to view past orders.
- 7. Manage saved addresses, notifications, and personal information from the Profile tab.

7.2 Technical Documentation Summary

This document has covered the system architecture (Flutter + BLoC/Cubit + Firebase), the simplified database schema (Users, Addresses, Products, Cart_Products, Fav_Products, Orders, Notifications), and API/integration notes (Firebase Authentication and Cloud Firestore as the primary backend services, Dio reserved for any external REST integrations). Refer to Section 4 for full architecture, ERD, and diagrams, and Section 5 for the source code structure.

7.3 Project Presentation (PPT/PDF)

Prepare a short slide deck summarizing: the problem and motivation, key features (auth, catalog/search, cart & checkout, tracking, profile), architecture overview, a live or recorded demo, challenges faced (e.g., synchronizing cart totals with Firestore, managing multiple Cubit states cleanly), and future improvements (payment gateway integration, admin dashboard, push notifications).

7.4 Video Demonstration (Optional)

Record a short (3-5 minute) screen capture walking through: sign up/login, browsing and searching products, adding to cart, checkout, and order tracking, to accompany the written submission.